

Product & Tech Generalist Assessment

Candidate: Debojyoti Dey Role: Product Engineer GitHub: github.com/its-debojyoti-dey/matchmaker-copilot

Part 1: Diagnose the Problem

(Word Count: ~310 words)

1. Three Questions to Investigate

1. Why are 35% of shared profiles violating clients' explicit deal-breakers?

Out of 690 rejections, 241 shared profiles (~24% of all dispatches) were dead on arrival. We need to know if this comes from cognitive fatigue, clunky CRM views where deal-breakers sit on separate tabs, or lack of pre-send validation. Fixing this claws back nearly a quarter of all wasted matchmaker effort immediately.

2. What explains the 2x performance gap between Matchmaker A (44%) and B (21%)?

Matchmaking quality currently lives in individual heads rather than shared systems. Does A write better pitch notes, run an informal pre-screening checklist, or hold fewer clients? If we find the specific workflow differences, we can turn A's habits into standard software guardrails for everyone.

3. Where do clients' stated preferences diverge from who they actually accept?

The prompt notes that clients often reject a profile early on, only to accept a very similar one later. If we treat every stated preference as an absolute filter, we starve our own pipeline. We need to distinguish true hard stops (smoking, kids) from flexible ideals (height, specific hobbies).

2. The Biggest Problem in the Funnel

The drop from **Profiles Shared (1,000)** to **Profiles Accepted (310)** is our biggest leak. That is a 69% loss right at the start.

Assumption: Downstream friction (contact exchange drops, date cancellations) happens partly because clients had lukewarm conviction from day one. Sending candidates who breach stated deal-breakers damages trust immediately. Fixing match quality upstream gives us the biggest return down the line. Every genuinely good recommendation feeds conversations and completed meetings.

3. Three Metrics to Track

1. Preventable Rejection Rate (PRR):

$$\text{PRR} = (\text{Rejections due to stated deal-breakers} / \text{Total Rejections}) * 100$$

Tracks basic operational discipline. Benchmark: drop from 35% to under 5%.

2. Profile Acceptance Rate (PAR):

$$\text{PAR} = (\text{Profiles Accepted} / \text{Profiles Shared}) * 100$$

Measures recommendation quality across the team. Benchmark: lift from 31% to over 42%.

3. Search Time per Match (STpM):

Hours matchmakers spend per client each week hunting for profiles. Currently at 2 hours. Benchmark: cut to under 30 minutes so matchmakers can spend their time talking to clients instead of filtering databases.

Problem

Matchmakers spend 2 hours per client each week manually cross-referencing candidate profiles against preference sheets. They miss stated deal-breakers under cognitive load, causing 35% of rejections. On top of that, rejection feedback arrives as free-text emails and disappears into inboxes without improving future searches.

User

Internal matchmakers (specifically bringing operators like Matchmaker B up to A's standard) and the operations lead.

Solution: Matchmaker Pre-Flight Auditor & Feedback Engine

A lightweight internal tool built directly into the profile dispatch flow:

- 1. Pre-flight deal-breaker check:** When a matchmaker pairs a candidate with a client, the tool runs a deterministic check on hard constraints: smoking, drinking, plans for children, age limits, location, and diet. If any deal-breaker fails, the dispatch button disables and shows a plain warning: *"Blocked: Client specified non-smoker. Candidate smokes socially."* The matchmaker cannot send the email without an explicit reason logged.
- 2. Contextual pitch helper:** Once hard constraints clear, an LLM compares the client's qualitative notes with the candidate's bio (values, communication style, shared interests). It drafts a 2-sentence rationale for the matchmaker to edit and paste into the client email.
- 3. Structured rejection parser:** When a client replies rejecting a profile, the matchmaker pastes the email text into the tool. The LLM extracts the core reason into set categories (`deal_breaker_missed` , `lifestyle_mismatch` , `aesthetic_preference`). If the rejection was a missed deal-breaker that bypassed the check, it flags an ops alert. If it reveals an unstated preference, it updates the client's profile tag list.

Data

Client profile: hard deal-breakers (booleans and ranges), soft preferences (free tags), historical accept/reject logs.

Candidate profile: demographic fields, lifestyle tags, written bio.

Inbound feedback: raw rejection text from client emails.

Technology

- **Frontend:** Next.js 14 (App Router) + Tailwind CSS + Lucide React, featuring responsive pass/fail badges, tabbed matchmaker workflow, and real-time validation states.
- **Rules Engine:** Domain-Driven Design (DDD) composable rule pipeline with pure TypeScript interfaces (`MatchRule` , `STANDARD_MATCH_RULES`), running deterministic deal-breaker checks at zero latency.
- **AI & NLP:** Dual-mode architecture: zero-config heuristic classifier for instant offline evaluation, plus Next.js API routes integrated with Gemini 3.8 Flash (Gemini 3.7 Flash fallback) via structured JSON outputs.
- **Testing & Deployment:** Node.js native test runner (`node:test`) for unit-testing deal-breaker edge cases, deployed on Vercel with GitHub repository integration.

Success Metric

Primary: Preventable Rejection Rate drops from 35% to under 5% in 14 days.

Secondary: Profile Acceptance Rate climbs from 31% to over 40%, and matchmaker search time drops by at least half (saving ~1 hour per client weekly).

Part 3: Prototype Implementation

The working prototype demonstrates the full matchmaking dispatch and feedback cycle:

1. **Interactive Client & Candidate Selector:** Evaluates curated test pairs (Priya Sharma against smoking, children, and age boundary mismatches vs. 100% clean matches).
2. **Deterministic Pre-Flight Gatekeeper:** Pluggable rule pipeline that checks hard constraints and strictly disables dispatch upon deal-breaker violations.
3. **AI Pitch Generator:** Auto-generates personalized 2-sentence client pitch notes for approved candidates.
4. **Rejection Feedback Intelligence:** Ingests raw client rejection emails, categorizes root causes into structured tags (`deal_breaker_missed` , `revealed_preference` , `aesthetic_vibe`), and flags operator accountability.
5. **Funnel Economics Model:** Models the compounding impact of eliminating preventable rejections on downstream completed meetings (projected +81% lift from 42 to 76 completed dates).
6. **Automated Unit Tests:** 6/6 automated test suites passing in 103ms verifying all critical failure and pass conditions.

Live Codebase & Demo: <https://github.com/its-debojyoti-dey/matchmaker-copilot>

Part 4: One Curveball

(Word Count: ~180 words)

Scenario: The metric does not move after two weeks.

1. What to check first

Adoption. Did matchmakers actually run recommendations through the tool, or did they work around it to hit their numbers? Check dispatch logs and total profiles sent.

2. What data to look at

- **Preventable Rejection Rate:** Did PRR drop from 35% to under 5%? If PRR dropped but overall acceptance stayed flat at 31%, our hygiene check worked, but revealed that stated deal-breakers were just polite cover for other objections.
- **Rejection category shift:** Check the new structured rejection data. What replaced deal-breakers? Look for spikes in photo reactions, career expectations, or vague vibe complaints.
- **Matchmaker variance:** Did Matchmaker B improve while Matchmaker A stayed unchanged?

3. Iterate, change, or kill?

Iterate. Do not kill. Blocking deal-breaker violations is basic hygiene. A curated service cannot send smokers to non-smokers and keep credibility.

However, change the focus for Sprint 2: pivot from stated filters to revealed preferences. That means analyzing the candidates clients actually accept, tightening photo selection standards, and coaching clients on flexible criteria.

AI Usage Statement

- **Tools used:** Antigravity, Cursor, Gemini 3.8 Flash.
- **What I used them for:** Scaffolding the pluggable `MatchRule` pipeline, generating realistic edge-case matrimonial profiles, and drafting native Node test runner test suites.
- **Where I disagreed with the AI:** The AI initially recommended building a complex vector database (RAG) with embedding cosine similarity to rank profiles. I rejected this approach. The data proved that 35% of rejections came from simple boolean deal-breaker failures (smoking, children, age limits). Replacing deterministic business rules with probabilistic vector search would have introduced hallucinations, obscured auditability, and overcomplicated what is fundamentally a checklist problem.

The Date Crew — Product Engineer Assessment Candidate: Debojyoti Dey Repository: [its-debojyoti-dey/matchmaker-copilot](https://github.com/debojyoti-dey/matchmaker-copilot)